

Persistent Memory: The Historical Detection Signal

Overview

The drift detector runs two independent detection signals in parallel, not one. Alongside the statistical baseline comparison (Mahalanobis distance against per-route sub-clusters), a second, entirely separate mechanism permanently remembers every confirmed drift prompt and checks new traffic against that memory directly, via similarity search. This page documents why a second signal exists, how it works, and where its current limits are.

Why a Single Detection Signal Isn't Enough

The statistical branch is good at catching **novel** drift — traffic that looks meaningfully different from established normal behavior, even if it's never been seen before. But it has a structural weakness: it depends entirely on the current state of the baseline. If the baseline is mid-recompute, degraded, or simply not sensitive enough to catch a particular kind of drift (see the Multi-Cluster Baseline page for a concrete example of this), a real attack can slip through the statistical check.

A confirmed attack should never depend on any of that. If we already know, with certainty, that a specific prompt is an attack — because it was already caught and confirmed once — there is no reason a repeat of that exact attack should ever require a fresh statistical judgment call to catch again.

How It Works

Every time a prompt is confirmed as drift, its embedding is written to a permanent store and never deleted. This store is checked independently, for every new incoming prompt, using nearest-neighbor similarity search — the same HNSW-backed infrastructure used elsewhere in the system (see HNSW Indexing & Retrieval). If a new prompt's embedding is sufficiently close to any confirmed-attack embedding, it is flagged immediately, regardless of what the statistical baseline currently says about it.

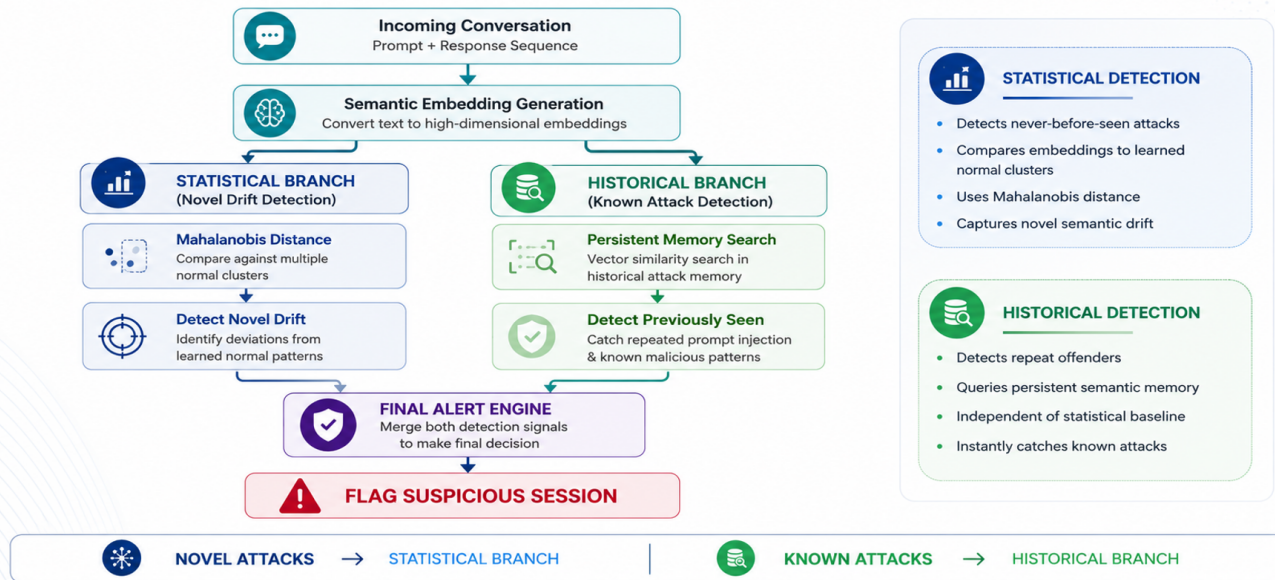
This is a genuinely different mechanism from the statistical branch, not a variation on it:

	Statistical branch	Historical branch
Detects	Novel drift	Repeated/near-repeated attacks
Depends on	Current baseline state	Nothing except its own stored memory
Mechanism	Diagonal Mahalanobis distance	Nearest-neighbor cosine similarity
Recomputed	Periodically (baseline job)	Never invalidated — grows permanently

A session is flagged if **either** signal trips.

HOW DETECTION ACTUALLY WORKS

Two independent detection signals work in parallel



Threshold and Verification

The similarity threshold for a match is currently **0.9** cosine similarity. This was verified directly: an exact repeat of a previously confirmed attack is caught instantly, with similarity scoring 1.0, entirely independent of whether or when the statistical baseline was last recomputed.

Known Limitation

At a 0.9 threshold, this signal reliably catches **near-exact repeats** of a confirmed attack, but not loosely-worded paraphrases of the same underlying attack. A meaningfully reworded version of a known attack may not cross the similarity threshold, and would need to be caught by the statistical branch instead, if at all.

Lowering the threshold to catch more paraphrases is not a safe change to make by intuition — it needs a proper precision/recall sweep against labeled data first, the same way the statistical branch's sigma threshold was tuned (see the Sigma Threshold Tuning page), rather than an untested guess. This is tracked as a future enhancement, not yet implemented.

Why This Matters Architecturally

This is a good example of defense in depth applied to detection logic specifically: two mechanisms, covering two different failure modes, neither one a substitute for the other. A system relying solely on the statistical branch would be vulnerable during baseline gaps; a system relying solely on historical memory would never catch anything genuinely new. Running both, and flagging on either, closes that gap without waiting for a single "smarter" algorithm to cover both cases at once.